



A01 - Funciones objetivo Algoritmos Metaheurísticos

José Luis Chávez Gómez

1 de septiembre de 2026

1. Marco Teórico

En el contexto de los algoritmos metaheurísticos (algoritmos genéticos, optimización por enjambre de partículas, recocido simulado, entre otros), una **función objetivo** es la función matemática que un algoritmo de optimización busca minimizar o maximizar. Formalmente, dada una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$, el problema de optimización consiste en encontrar el vector $x^* \in \mathbb{R}^d$ tal que

$$x^* = \arg \min_{x \in \Omega} f(x) \quad (1)$$

donde $\Omega \subseteq \mathbb{R}^d$ es el espacio de búsqueda (dominio) permitido para las variables de decisión.

Para evaluar y comparar el desempeño de distintos algoritmos metaheurísticos, la comunidad científica utiliza un conjunto estandarizado de **funciones de prueba** (benchmark functions), cuya solución óptima (x^* y $f(x^*)$) es conocida de antemano. Esto permite medir qué tan cerca llega un algoritmo al óptimo real, y bajo qué condiciones (número de evaluaciones, dimensionalidad, etc.).

Estas funciones se agrupan comúnmente según la forma de su "paisaje" de búsqueda:

- **Funciones tipo tazón (unimodales):** presentan un único mínimo global y ninguna estructura engañosa. Ejemplos: Sphere, Sum of Squares.
- **Funciones tipo valle:** unimodales, pero con un valle estrecho y curvo que dificulta la convergencia exacta aunque el valle mismo sea fácil de encontrar. Ejemplo: Rosenbrock.
- **Funciones con muchos mínimos locales (multimodales):** contienen numerosos óptimos locales que pueden atrapar a un algoritmo antes de llegar al óptimo global. Ejemplos: Rastrigin, Ackley, Schwefel.

- **Funciones con fuerte interacción entre variables:** el valor de la función no puede descomponerse en contribuciones independientes por dimensión. Ejemplos: Zakharov, las funciones Perm.

El presente reporte implementa y visualiza 18 funciones objetivo estándar, todas capaces de recibir un vector de n dimensiones y regresar un único valor escalar, tal como lo requiere cualquier algoritmo metaheurístico de propósito general.

2. Funciones Objetivo

2.1. Función Sphere

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d x_i^2 \quad (2)$$

Dominio: $x_i \in [-5,12,5,12]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Sphere(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-5.12] * dim
8         lim_sup = [5.12] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         sum = 0
13         for xi in x:
14             sum = sum + (xi ** 2)
15         return sum
16
17 obj = Sphere(dim=2)
18 obj.draw3d(name="Sphere")
19 plt.savefig("sphere_3d.png")
20 plt.show()

```

Gráfica 3D:

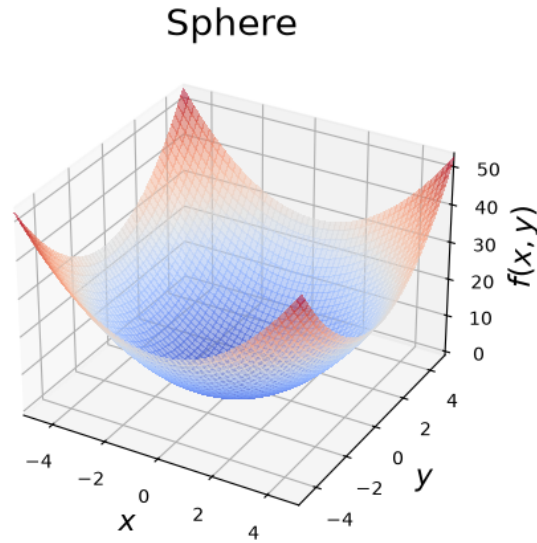


Figura 1: Superficie de la función Sphere en 2 dimensiones de entrada.

2.2. Función Rastrigin

Fórmula:

$$f(\mathbf{x}) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)] \quad (3)$$

Dominio: $x_i \in [-5,12,5,12]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Rastrigin(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-5.12] * dim
8         lim_sup = [5.12] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         sum = 0
14         for xi in x:
15             sum = sum + ((xi ** 2) - (10 * np.cos(2*np.pi*xi)))
16         result = (10*n) + sum
17         return result
18
19 obj = Rastrigin(dim=2)
20 obj.draw3d(name="Rastrigin")
21 plt.savefig("rastrigin_3d.png")
22 plt.show()
23
24 print(obj.function([0, 0]))
25 print(obj.function([1, 1]))

```

Gráfica 3D:

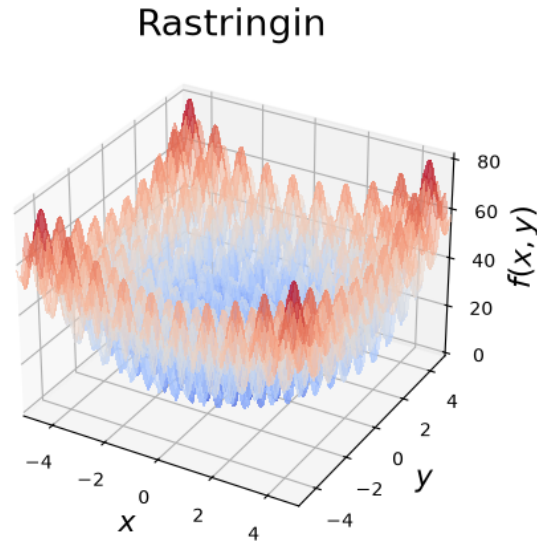


Figura 2: Superficie de la función Rastrigin en 2 dimensiones de entrada.

2.3. Función Ackley

Fórmula:

$$f(\mathbf{x}) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(c x_i) \right) + a + e \quad (4)$$

Dominio: $x_i \in [-32,768, 32,768]$ ($a = 20$, $b = 0,2$, $c = 2\pi$)

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Ackley(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-32.768] * dim
8         lim_sup = [32.768] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         a = 20
14         b = 0.2
15         c = 2*np.pi
16         sum1 = 0
17         sum2 = 0
18
19         for xi in x:
20             sum1 = sum1 + (xi ** 2)
21             sum2 = sum2 + np.cos(c * xi)
22
```

```

23     part1 = np.sqrt(sum1 / n)
24     part2 = sum2 / n
25
26     result = -a * np.exp(-b * part1) - np.exp(part2) + a + np.e
27     return result
28
29
30 obj = Ackley(dim=2)
31 obj.draw3d(name="Ackley")
32 plt.savefig("ackley_3d.png")
33 plt.show()
34
35 print(obj.function([0,0]))

```

Gráfica 3D:

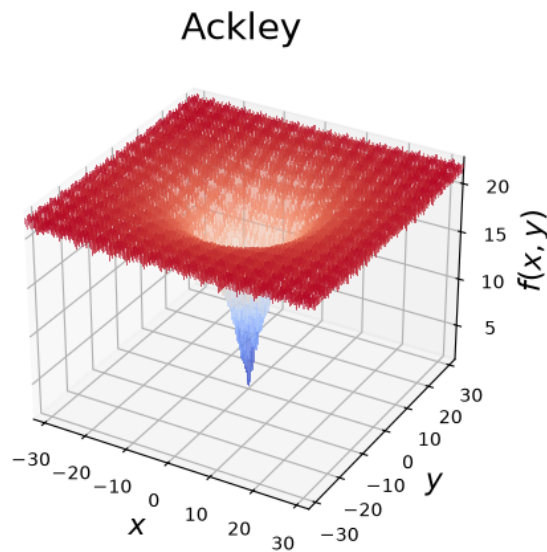


Figura 3: Superficie de la función Ackley en 2 dimensiones de entrada.

2.4. Función Levy

Fórmula:

$$f(\mathbf{x}) = \sin^2(\pi w_1) + \sum_{i=1}^{d-1} (w_i - 1)^2 [1 + 10 \sin^2(\pi w_i + 1)] + (w_d - 1)^2 [1 + \sin^2(2\pi w_d)], \quad w_i = 1 + \frac{x_i - 1}{4} \quad (5)$$

Dominio: $x_i \in [-10, 10]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Levy(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-10] * dim
8         lim_sup = [10] * dim

```

```

9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         w = [0] * n
14         for i in range(n):
15             w[i] = 1 + ((x[i] - 1) / 4)
16         term1 = np.sin(np.pi * w[0]) ** 2
17         sum_mid = 0
18         for i in range(n-1):
19             wi = w[i]
20             parte = ((wi - 1) ** 2) * (1 + 10 * (np.sin(np.pi * wi + 1) ** 2))
21             sum_mid = sum_mid + parte
22         w_last = w[n-1]
23         term3 = ((w_last - 1) ** 2) * (1 + (np.sin(2*np.pi*w_last) ** 2))
24         result = term1 + sum_mid + term3
25         return result
26
27 obj = Levy(dim=2)
28 obj.draw3d(name="Levy")
29 plt.savefig("levy_3d.png")
30 plt.show()
31
32 print(obj.function([1, 1]))
33 print(obj.function([0, 0]))

```

Gráfica 3D:

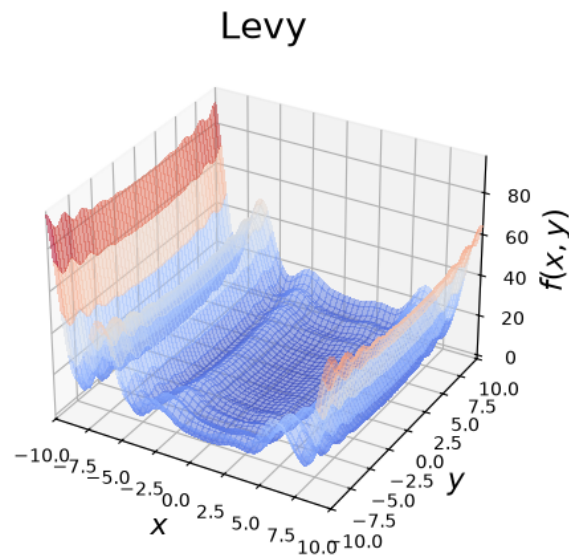


Figura 4: Superficie de la función Levy en 2 dimensiones de entrada.

2.5. Función Schwefel

Fórmula:

$$f(\mathbf{x}) = 418,9829 d - \sum_{i=1}^d x_i \sin(\sqrt{|x_i|}) \quad (6)$$

Dominio: $x_i \in [-500, 500]$

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Schwefel(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-500] * dim
8         lim_sup = [500] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         d = len(x)
13         sum = 0
14         for xi in x:
15             sum = sum + (xi * np.sin(np.sqrt(np.abs(xi))))
16         result = (418.9829 * d) - sum
17         return result
18
19 obj = Schwefel(dim=2)
20 obj.draw3d(name="Schwefel")
21 plt.savefig("schwefel_3d.png")
22 plt.show()
23
24 print(obj.function([1, 1]))
25 print(obj.function([0, 0]))
26 print(obj.function([420.9687, 420.9687]))
```

Gráfica 3D:

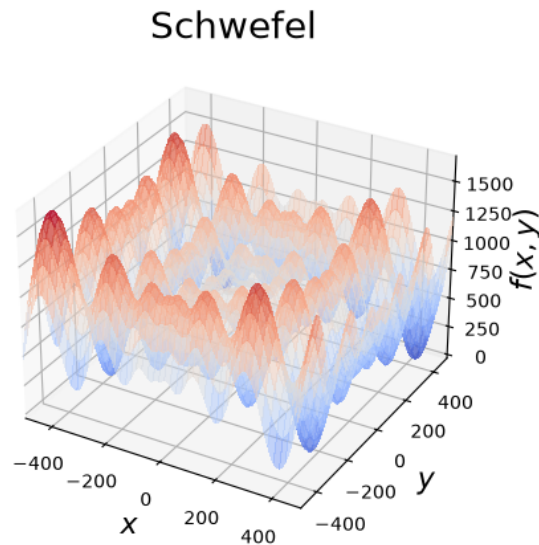


Figura 5: Superficie de la función Schwefel en 2 dimensiones de entrada.

2.6. Función Rotated Hyperellipsoid

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d \left(\sum_{j=1}^i x_j \right)^2 \quad (7)$$

Dominio: $x_i \in [-65,536, 65,536]$

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Rotated_Hyperellipsoid(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-65.536] * dim
8         lim_sup = [65.536] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         sum_ext = 0
14         for i in range(0, n):
15             sum_int = 0
16             for j in range(0, i + 1):
17                 sum_int = sum_int + x[j]
18             sum_ext = sum_ext + (sum_int ** 2)
19         return sum_ext
20
21 obj = Rotated_Hyperellipsoid(dim=2)
22 obj.draw3d(name="Rotated Hyperellipsoid")
23 plt.savefig("rotated_hyperellipsoid_3d.png")
24 plt.show()
25
26 print(obj.function([1, 1]))
27 print(obj.function([0, 0]))
28 print(obj.function([1, 2]))
```

Gráfica 3D:

Rotated Hyperellipsoid

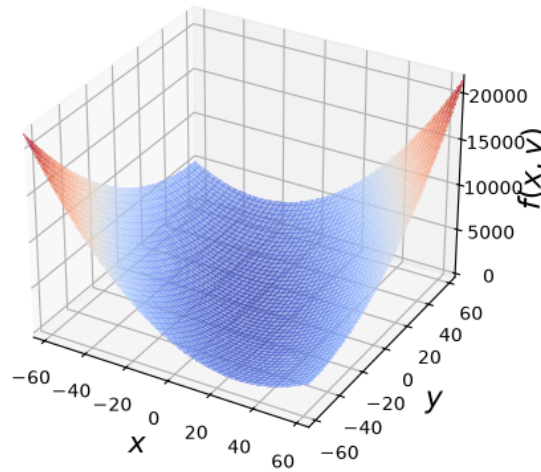


Figura 6: Superficie de la función Rotated Hyperellipsoid en 2 dimensiones de entrada.

2.7. Función Rosenbrock

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^{d-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2] \quad (8)$$

Dominio: $x_i \in [-5, 10]$

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Rosenbrock(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-5] * dim
8         lim_sup = [10] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         sum = 0
14         for i in range(n-1):
15             xi = x[i]
16             x_sig = x[i + 1]
17             term = (100 * (x_sig - (xi ** 2)) ** 2) + ((xi - 1) ** 2)
18             sum = sum + term
19         return sum
20
21 obj = Rosenbrock(dim=2)
22 obj.draw3d(name="Rosenbrock")
23 plt.savefig("rosenbrock_3d.png")
24 plt.show()
25
26 print(obj.function([1, 1]))
```

```
27 print(obj.function([0, 0]))
```

Gráfica 3D:

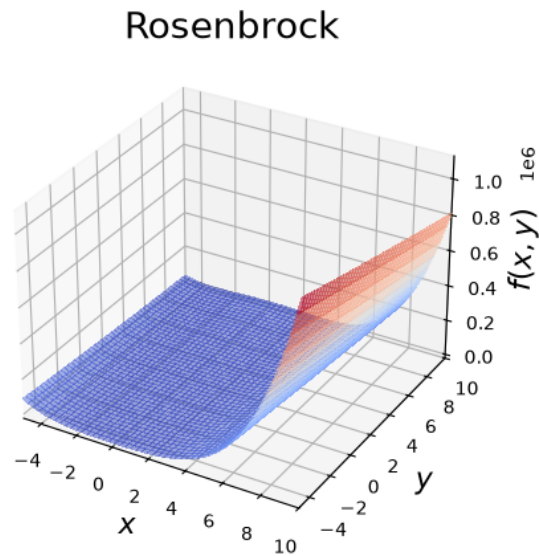


Figura 7: Superficie de la función Rosenbrock en 2 dimensiones de entrada.

2.8. Función Griewank

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d \frac{x_i^2}{4000} - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1 \quad (9)$$

Dominio: $x_i \in [-600, 600]$

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Griewank(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-600] * dim
8         lim_sup = [600] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         sum = 0
13         product = 1
14         for i, xi in enumerate(x, start=1):
15             sum = sum + ((xi ** 2) / 4000)
16             product = product * np.cos(xi / np.sqrt(i))
17         result = sum - product + 1
18         return result
19
20 obj = Griewank(dim=2)
21 obj.draw3d(name="Griewank")
22 plt.savefig("griewank_3d.png")
```

```

23 plt.show()
24
25 print(obj.function([1, 0]))
26 print(obj.function([0, 0]))

```

Gráfica 3D:

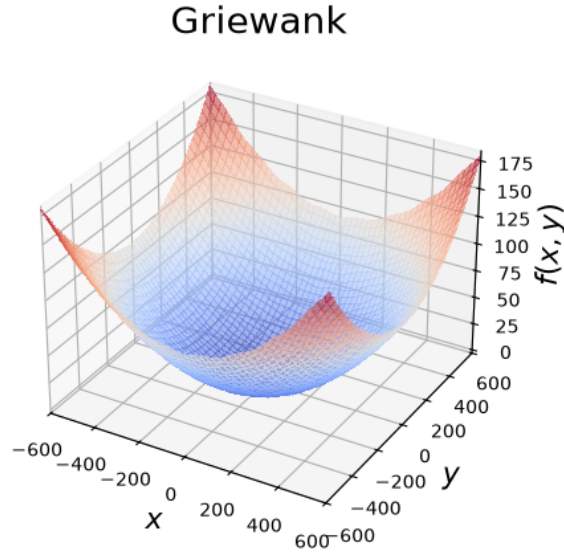


Figura 8: Superficie de la función Griewank en 2 dimensiones de entrada.

2.9. Función Perm 0, d , β

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d \left[\sum_{j=1}^d (j + \beta) \left(x_j^i - \left(\frac{1}{j} \right)^i \right) \right]^2 \quad (10)$$

Dominio: $x_i \in [-d, d]$ ($\beta = 10$)

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Perm0db(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-dim] * dim
8         lim_sup = [dim] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         d = len(x)
13         beta = 10
14         sum_ext = 0
15         for i in range(1, d + 1):
16             sum_int = 0
17             for j in range(1, d + 1):
18                 xj = x[j - 1]

```

```

19         term = (j + beta) * ((xj ** i) - ((1/j) ** i))
20         sum_int = sum_int + term
21         sum_ext = sum_ext + (sum_int ** 2)
22         return sum_ext
23
24 obj = Perm0db(dim=2)
25 obj.draw3d(name="Perm0db")
26 plt.savefig("perm0db_3d.png")
27 plt.show()
28
29 print(obj.function([1, 0.5]))
30 print(obj.function([0, 0]))

```

Gráfica 3D:

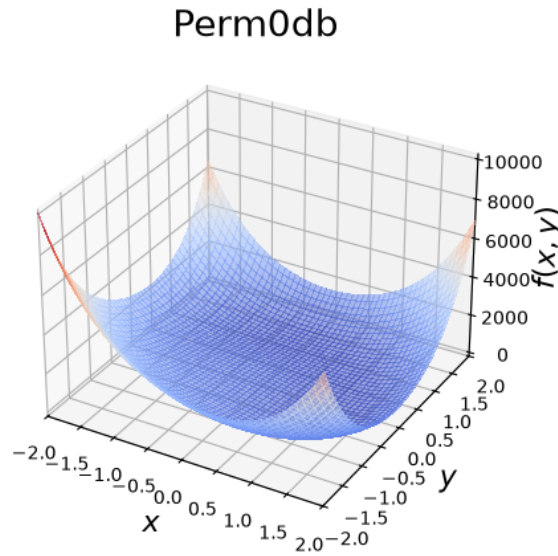


Figura 9: Superficie de la función Perm 0, d, β en 2 dimensiones de entrada.

2.10. Función Sum of Different Powers

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d |x_i|^{i+1} \quad (11)$$

Dominio: $x_i \in [-1, 1]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Sum_differentpowers(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-1] * dim
8         lim_sup = [1] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):

```

```

12     sum = 0
13     for i, xi in enumerate(x, start=1):
14         sum = sum + abs(xi) ** (i + 1)
15     return sum
16
17 obj = Sum_differentpowers(dim=2)
18 obj.draw3d(name="Sum of different powers")
19 plt.savefig("sumofdifferentpowers_3d.png")
20 plt.show()
21
22 print(obj.function([1, 1]))
23 print(obj.function([0, 0]))

```

Gráfica 3D:

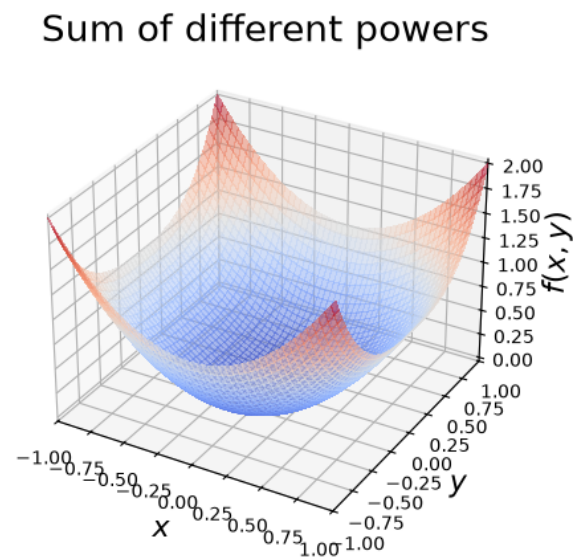


Figura 10: Superficie de la función Sum of Different Powers en 2 dimensiones de entrada.

2.11. Función Sum of Squares

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d i x_i^2 \quad (12)$$

Dominio: $x_i \in [-10, 10]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Sum_squares(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-10] * dim
8         lim_sup = [10] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):

```

```

12     sum = 0
13     for i, xi in enumerate(x, start=1):
14         sum = sum + i * (xi ** 2)
15     return sum
16
17 obj = Sum_squares(dim=2)
18 obj.draw3d(name="Sum of Squares")
19 plt.savefig("sumofsquares_3d.png")
20 plt.show()
21
22 print(obj.function([1, 1]))
23 print(obj.function([0, 0]))

```

Gráfica 3D:

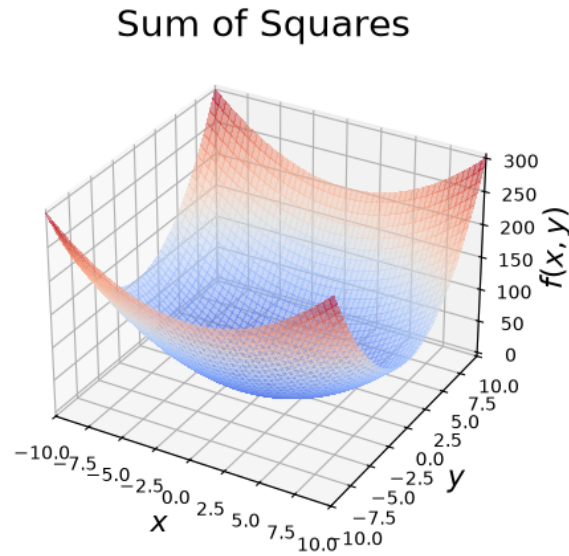


Figura 11: Superficie de la función Sum of Squares en 2 dimensiones de entrada.

2.12. Función Trid

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d (x_i - 1)^2 - \sum_{i=2}^d x_i x_{i-1} \quad (13)$$

Dominio: $x_i \in [-d^2, d^2]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Trid(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-(dim ** 2)] * dim
8         lim_sup = [(dim ** 2)] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):

```

```

12     n = len(x)
13     sum1 = 0
14     for xi in x:
15         sum1 = sum1 + ((xi - 1) ** 2)
16     sum2 = 0
17     for i in range(1, n):
18         xi = x[i]
19         x_ant = x[i - 1]
20         sum2 = sum2 + (xi * x_ant)
21     result = sum1 - sum2
22     return result
23
24 obj = Trid(dim=2)
25 obj.draw3d(name="Trid")
26 plt.savefig("trid_3d.png")
27 plt.show()
28
29 print(obj.function([2, 2]))
30 print(obj.function([0, 0]))

```

Gráfica 3D:

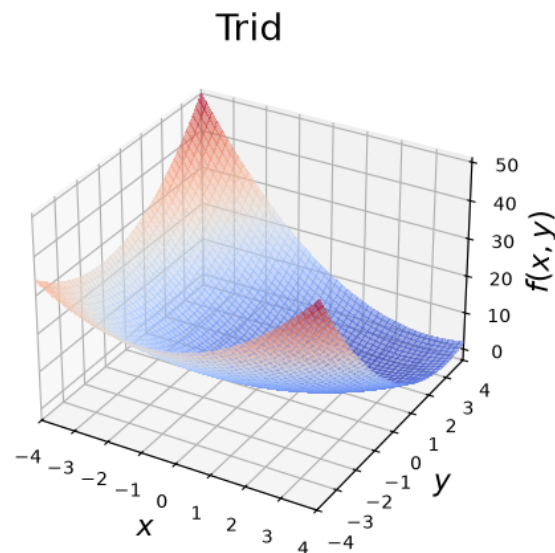


Figura 12: Superficie de la función Trid en 2 dimensiones de entrada.

2.13. Función Zakharov

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d x_i^2 + \left(\sum_{i=1}^d 0,5 i x_i \right)^2 + \left(\sum_{i=1}^d 0,5 i x_i \right)^4 \quad (14)$$

Dominio: $x_i \in [-5, 10]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4

```

```

5 class Zakharov(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-5] * dim
8         lim_sup = [10] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         sumA = 0
13         sumB = 0
14         for i, xi in enumerate(x, start=1):
15             sumA = sumA + (xi ** 2)
16             sumB = sumB + (0.5 * i * xi)
17         result = sumA + (sumB ** 2) + (sumB ** 4)
18         return result
19
20 obj = Zakharov(dim=2)
21 obj.draw3d(name="Zakharov")
22 plt.savefig("zakharov_3d.png")
23 plt.show()
24
25 print(obj.function([1, 0]))
26 print(obj.function([0, 0]))

```

Gráfica 3D:

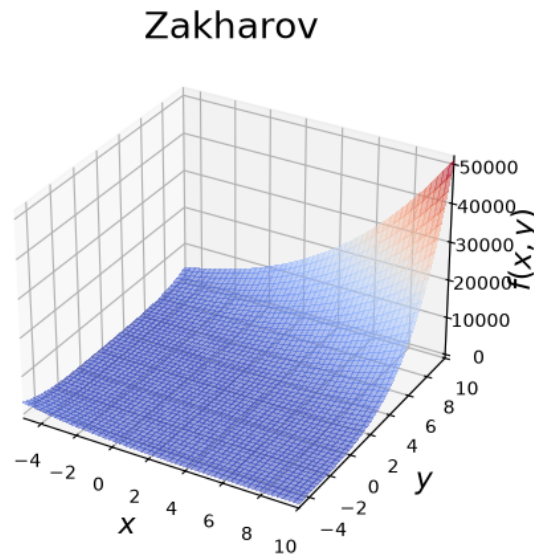


Figura 13: Superficie de la función Zakharov en 2 dimensiones de entrada.

2.14. Función Dixon-Price

Fórmula:

$$f(\mathbf{x}) = (x_1 - 1)^2 + \sum_{i=2}^d i (2x_i^2 - x_{i-1})^2 \quad (15)$$

Dominio: $x_i \in [-10, 10]$

Código:

```

1 from basics import Objective_function

```

```

2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Dixon_Price(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-10] * dim
8         lim_sup = [10] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         first_term = ((x[0] - 1) ** 2)
14         sum = 0
15         for i in range(1, n):
16             xi = x[i]
17             x_ant = x[i-1]
18             index_formula = i + 1
19             term = index_formula * (((2*(xi ** 2)) - x_ant) ** 2)
20             sum = sum + term
21         result = first_term + sum
22         return result
23
24 obj = Dixon_Price(dim=2)
25 obj.draw3d(name="Dixon Price")
26 plt.savefig("dixon_price_3d.png")
27 plt.show()
28
29 print(obj.function([1, 1]))
30 print(obj.function([0, 0]))

```

Gráfica 3D:

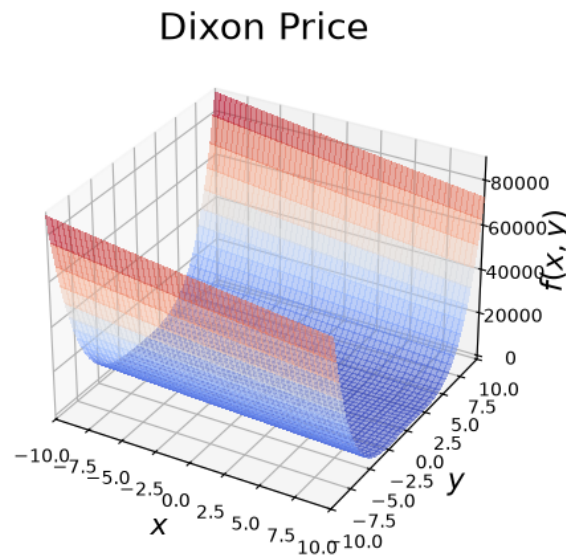


Figura 14: Superficie de la función Dixon-Price en 2 dimensiones de entrada.

2.15. Función Michalewicz

Fórmula:

$$f(\mathbf{x}) = - \sum_{i=1}^d \sin(x_i) \left[\sin\left(\frac{i x_i^2}{\pi}\right) \right]^{2m} \quad (16)$$

Dominio: $x_i \in [0, \pi]$ ($m = 10$)

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Michelewicz(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [0] * dim
8         lim_sup = [np.pi] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         m = 10
13         sum = 0
14         for i, xi in enumerate(x, start=1):
15             parte_int = np.sin(i * ((xi ** 2) / np.pi))
16             term = np.sin(xi) * (parte_int ** (2*m))
17             sum = sum + term
18         result = -sum
19         return result
20
21 obj = Michelewicz(dim=2)
22 obj.draw3d(name="Michelewicz")
23 plt.savefig("michelewicz_3d.png")
24 plt.show()
25
26 print(obj.function([0, 0]))
27 print(obj.function([1, 1]))
```

Gráfica 3D:

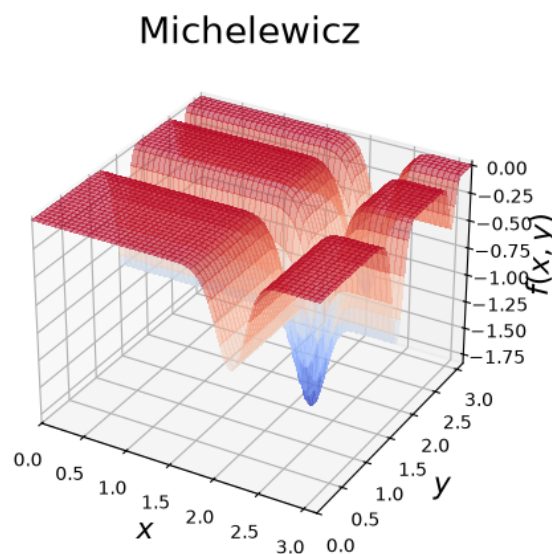


Figura 15: Superficie de la función Michalewicz en 2 dimensiones de entrada.

2.16. Función Perm d, β

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^d \left[\sum_{j=1}^d (j^i + \beta) \left(\left(\frac{x_j}{j} \right)^i - 1 \right) \right]^2 \quad (17)$$

Dominio: $x_i \in [-d, d]$ ($\beta = 0,5$)

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Permdb(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-dim] * dim
8         lim_sup = [dim] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         d = len(x)
13         beta = 0.5
14         sum_ext = 0
15         for i in range(1, d + 1):
16             sum_int = 0
17             for j in range(1, d + 1):
18                 xj = x[j - 1]
19                 term = ((j ** i) + beta) * (((xj / j) ** i) - 1)
20                 sum_int = sum_int + term
21             sum_ext = sum_ext + (sum_int ** 2)
22         return sum_ext
23
24 obj = Permdb(dim=2)
25 obj.draw3d(name="Permdb")
26 plt.savefig("permdb_3d.png")
27 plt.show()
28
29 print(obj.function([1, 2]))
30 print(obj.function([0, 0]))
```

Gráfica 3D:

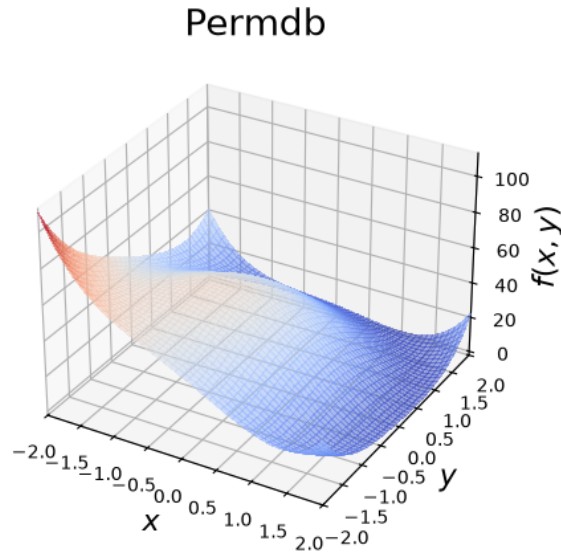


Figura 16: Superficie de la función Perm d, β en 2 dimensiones de entrada.

2.17. Función Styblinski-Tang

Fórmula:

$$f(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^d (x_i^4 - 16x_i^2 + 5x_i) \quad (18)$$

Dominio: $x_i \in [-5, 5]$

Código:

```

1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Styblinski(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-5] * dim
8         lim_sup = [5] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         sum = 0
13         for xi in x:
14             sum = sum + ((xi ** 4) - (16 * (xi ** 2)) + (5 * xi))
15         result = 0.5 * sum
16         return result
17
18 obj = Styblinski(dim=2)
19 obj.draw3d(name="Styblinski")
20 plt.savefig("styblinski_3d.png")
21 plt.show()
22
23 print(obj.function([1, 1]))
24 print(obj.function([0, 0]))

```

Gráfica 3D:

Styblinski

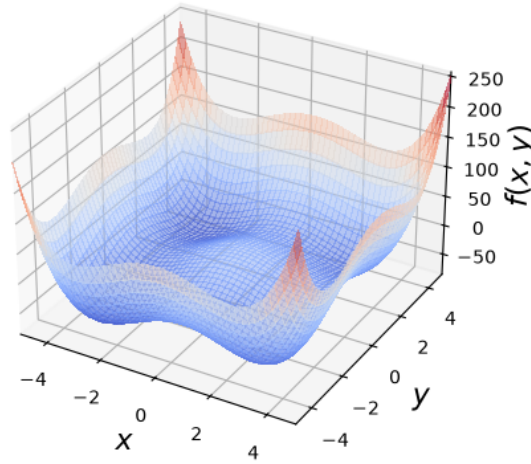


Figura 17: Superficie de la función Styblinski-Tang en 2 dimensiones de entrada.

2.18. Función Bohachevsky

Fórmula:

$$f(\mathbf{x}) = \sum_{i=1}^{d-1} [x_i^2 + 2x_{i+1}^2 - 0,3 \cos(3\pi x_i) - 0,4 \cos(4\pi x_{i+1}) + 0,7] \quad (19)$$

Dominio: $x_i \in [-100, 100]$

Código:

```
1 from basics import Objective_function
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class Bohachevsky(Objective_function):
6     def __init__(self, dim):
7         lim_inf = [-100] * dim
8         lim_sup = [100] * dim
9         super().__init__(dim, lim_inf, lim_sup)
10
11     def function(self, x):
12         n = len(x)
13         sum = 0
14         for i in range(n-1):
15             xi = x[i]
16             x_sig = x[i + 1]
17             term = (xi ** 2) + (2*(x_sig ** 2)) - 0.3*np.cos(3*np.pi*xi) - 0.4*
18                 np.cos(4*np.pi*x_sig) + 0.7
19             sum = sum + term
20         return sum
21
22 obj = Bohachevsky(dim=2)
23 obj.draw3d(name="Bohachevsky")
24 plt.savefig("bohachevsky_3d.png")
25 plt.show()
```

```

25
26 print(obj.function([1, 1]))
27 print(obj.function([0, 0]))

```

Gráfica 3D:

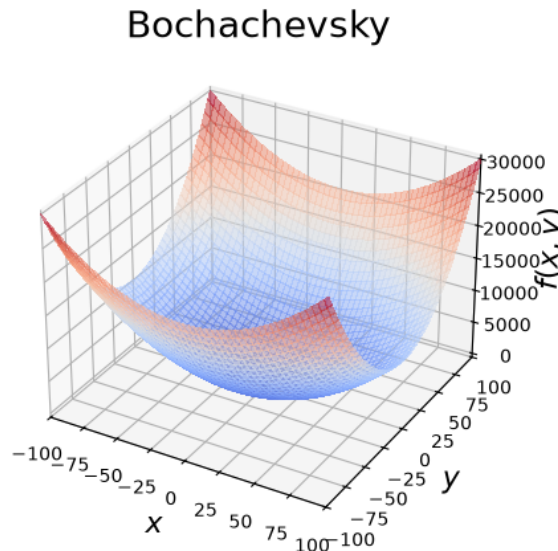


Figura 18: Superficie de la función Bohachevsky en 2 dimensiones de entrada.

3. Conclusión

La mayor fuente de errores durante la implementación no fue conceptual, sino de *manejo de índices*: la notación matemática habitual indexa los vectores a partir de $i = 1$, mientras que Python lo hace a partir de 0, y los límites de los rangos de suma ($\sum_{i=1}^d$ frente a `range(1, d+1)`) son una fuente frecuente de errores por un elemento (“off-by-one”). Este tipo de error se presentó de forma repetida al trabajar con Trid, Rotated Hyperellipsoid y las funciones Perm, y su corrección reforzó la importancia de validar cada función evaluándola en su mínimo global teórico antes de darla por correcta, en lugar de confiar únicamente en que el código *corra sin errores*.

También quedó claro que no todas las funciones objetivo representan el mismo tipo de desafío para un algoritmo metaheurístico. Las funciones unimodales y separables (Sphere, Sum of Squares) tienen un paisaje de búsqueda sencillo, donde casi cualquier estrategia de búsqueda converge sin dificultad. En contraste, las funciones multimodales (Rastrigin, Ackley, Griewank, Schwefel) están diseñadas deliberadamente para contener múltiples mínimos locales que pueden atrapar a un algoritmo antes de alcanzar el óptimo global —Schwefel es un caso particularmente ilustrativo, pues su mínimo se ubica lejos del origen, penalizando estrategias de búsqueda que asumen que el óptimo está cerca del centro del dominio. Por su parte, las funciones con fuerte interacción entre variables (Rosenbrock, Dixon-Price, Zakharov, las funciones Perm) no pueden optimizarse dimensión por dimensión de forma independiente, lo cual representa un reto distinto: el algoritmo debe coordinar cambios simultáneos en varias variables para progresar, incluso cuando el paisaje no tiene mínimos locales engañosos.

En conjunto, este ejercicio evidencia por qué la comunidad de optimización metaheurística recurre a un conjunto diverso de funciones de prueba en lugar de una sola: cada una expone

una dificultad estructural distinta (multimodalidad, no separabilidad, valles estrechos, dominios descentrados), y un algoritmo que funcione bien en una de ellas no necesariamente lo hará en las demás. Esta lista de 18 funciones constituye, por tanto, una base sólida para evaluar de forma más completa el desempeño de los algoritmos metaheurísticos que se implementen en etapas posteriores del curso.

Referencias

- [1] Simon, D. (2013). *Evolutionary Optimization Algorithms: Biologically-Inspired and Population-Based Approaches to Computer Intelligence*. John Wiley & Sons.
- [2] Yang, X.-S. (2010). *Nature-Inspired Metaheuristic Algorithms* (2nd ed.). Luniver Press.
- [3] Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. John Wiley & Sons.
- [4] Engelbrecht, A. P. (2007). *Computational Intelligence: An Introduction* (2nd ed.). John Wiley & Sons.
- [5] Surjanovic, S., & Bingham, D. (2013). *Virtual Library of Simulation Experiments: Test Functions and Datasets*. Simon Fraser University. Recuperado de <https://www.sfu.ca/~ssurjano/>